

# Modelagem Física

Modelo Analítico de Dados para E-commerce

## Sumário

|           |                                              |          |
|-----------|----------------------------------------------|----------|
| <b>1</b>  | <b>Introdução</b>                            | <b>3</b> |
| <b>2</b>  | <b>Sistema Gerenciador de Banco de Dados</b> | <b>3</b> |
| <b>3</b>  | <b>Arquitetura Física em Camadas</b>         | <b>3</b> |
| 3.1       | RAW . . . . .                                | 4        |
| 3.2       | CORE . . . . .                               | 4        |
| 3.3       | ANALYTICS . . . . .                          | 4        |
| 3.4       | Mapeamento dos Arquivos para a RAW . . . . . | 4        |
| <b>4</b>  | <b>Evidências para os Tipos Físicos</b>      | <b>5</b> |
| <b>5</b>  | <b>Dicionário Físico do CORE</b>             | <b>5</b> |
| <b>6</b>  | <b>Estratégia Inicial de Índices</b>         | <b>7</b> |
| <b>7</b>  | <b>Scripts e Operação</b>                    | <b>7</b> |
| <b>8</b>  | <b>Validação da Arquitetura</b>              | <b>8</b> |
| <b>9</b>  | <b>Conclusão</b>                             | <b>8</b> |
| <b>10</b> | <b>Referências</b>                           | <b>8</b> |

# 1 Introdução

Este documento apresenta a modelagem física do projeto *Modelo Analítico de Dados para E-commerce*, transformando o modelo lógico aprovado em uma estrutura implementável e reproduzível no PostgreSQL.

A modelagem física define:

- o sistema gerenciador de banco de dados;
- a organização do banco em camadas;
- os tipos de dados físicos;
- nulabilidade e valores gerados;
- chaves primárias e estrangeiras;
- restrições de domínio;
- regras de integridade;
- índices iniciais;
- scripts de criação e remoção;
- procedimentos de validação da arquitetura.

A solução utiliza uma arquitetura ELT organizada em três camadas:

**Dataset Olist → RAW → CORE → ANALYTICS → Power BI / SQL**

A camada RAW preserva os dados recebidos, enquanto o CORE materializa o modelo relacional validado. A camada ANALYTICS será construída posteriormente a partir do CORE para atender necessidades específicas de análise e consumo.

As decisões são documentadas por sua finalidade técnica e semântica, sem depender de identificadores internos utilizados no gerenciamento do projeto.

# 2 Sistema Gerenciador de Banco de Dados

O sistema gerenciador de banco de dados adotado é o **PostgreSQL 18**, utilizando sempre a revisão secundária mais recente disponível dentro dessa versão principal.

A escolha considera os seguintes critérios:

- **Modelo relacional:** suporte maduro a tabelas, relacionamentos, chaves primárias, chaves estrangeiras e restrições;
- **Integridade dos dados:** suporte nativo a PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, nulabilidade e transações;
- **Tipos físicos:** disponibilidade de tipos adequados para valores monetários, coordenadas geográficas, números inteiros e dados temporais;
- **Consultas analíticas:** suporte a junções, agregações, CTEs, funções de janela e demais recursos SQL necessários às análises;
- **Automação:** possibilidade de executar scripts SQL por linha de comando, permitindo validações reproduzíveis;
- **Ecossistema:** integração com Python, DBeaver, Power BI e outras ferramentas de análise;
- **Portabilidade e custo:** software livre, amplamente utilizado e sem dependência de licenciamento proprietário.

### 2.1 Política de versão

O projeto fixa a versão principal **PostgreSQL 18**, mas não uma revisão secundária específica.

Correções e atualizações compatíveis dentro da mesma versão principal podem ser utilizadas sem alteração do modelo físico, desde que não introduzam incompatibilidades com os scripts SQL versionados.

Segundo a política oficial de versões do PostgreSQL, versões principais recebem manutenção por aproximadamente cinco anos. O PostgreSQL 18 possui suporte previsto até novembro de 2030.

Uma futura migração para outra versão principal deverá ser precedida pela execução completa dos scripts de criação e validação.

### 2.2 Uso do DBeaver

O DBeaver é utilizado como cliente gráfico para interação com o PostgreSQL.

Suas funções no projeto incluem:

- criação e gerenciamento da conexão com o servidor;
- execução manual dos scripts SQL versionados;

- inspeção de schemas, tabelas, colunas e restrições;
- visualização de índices e relacionamentos;
- execução de consultas exploratórias;
- apoio à inspeção dos dados após a carga.

O DBeaver não substitui o PostgreSQL e não constitui a definição oficial da estrutura física.

A estrutura reproduzível do banco permanece representada pelos scripts SQL versionados.

Credenciais, senhas, portas locais e configurações particulares de conexão não fazem parte do repositório.

### 3 Arquitetura Física em Camadas

A arquitetura foi organizada em três schemas:

`raw → core → analytics`

Cada camada possui uma responsabilidade distinta.

#### 3.1 RAW

O schema `raw` recebe uma representação fiel dos nove arquivos CSV utilizados pelo projeto.

Existe uma tabela RAW correspondente a cada arquivo.

Os nomes originais das colunas são mantidos e seus valores são armazenados inicialmente como `text`.

Essa decisão evita que erros de conversão impeçam a ingestão do arquivo ou eliminem a representação do valor originalmente recebido.

Cada tabela RAW adiciona somente três campos técnicos:

- `_id_raw`: identificador técnico da ocorrência carregada;
- `_arquivo_origem`: identificação do arquivo que originou o registro;
- `_carregado_em`: instante em que o registro foi incorporado à camada RAW.

Os atributos oriundos do dataset permanecem anuláveis e não recebem restrições de negócio.

A chave técnica da RAW identifica a ocorrência armazenada no banco, mas não afirma unicidade sobre os dados da fonte.

Valores inconsistentes continuam registráveis nessa camada para que possam ser posteriormente identificados, analisados e tratados.

### 3.2 CORE

O schema `core` materializa o modelo relacional aprovado.

Ele contém nove tabelas:

- `cliente`;
- `pedido`;
- `item_pedido`;
- `produto`;
- `vendedor`;
- `pagamento`;
- `avaliacao`;
- `prefixo_cep`;
- `geolocalizacao`.

No fluxo entre RAW e CORE são realizadas:

- conversões de tipos;
- renomeações de atributos;
- verificações de domínio;
- validações de nulabilidade;
- deduplicações necessárias;
- aplicação de chaves;
- aplicação de integridade referencial.

O CORE constitui a fonte relacional íntegra para consultas e para a construção posterior das estruturas analíticas.

### 3.3 ANALYTICS

O schema `analytics` é criado inicialmente sem tabelas ou views.

Objetos analíticos somente deverão ser adicionados quando estiverem definidos:

- a granularidade da estrutura;
- a métrica ou necessidade analítica atendida;
- as regras de transformação;
- a frequência de atualização;
- a forma de consumo.

Essa camada poderá conter views, tabelas derivadas, marts ou outras estruturas destinadas ao consumo analítico.

Ferramentas como Power BI e consumidores SQL deverão utilizar preferencialmente a camada ANALYTICS quando existir uma estrutura apropriada para a análise desejada.

### 3.4 Fluxo físico dos dados

O fluxo previsto é:

1. leitura dos arquivos CSV;
2. carregamento dos registros na camada RAW;
3. transformação e validação dos dados;
4. carga das entidades relacionais no CORE;
5. geração posterior de estruturas derivadas no ANALYTICS.

A arquitetura caracteriza um processo ELT porque os dados são primeiramente carregados no banco em sua representação bruta e somente depois transformados dentro do ambiente de dados.

### 3.5 Mapeamento dos arquivos para a RAW

| Arquivo                               | Tabela RAW                            |
|---------------------------------------|---------------------------------------|
| olist_customers_dataset.csv           | raw.olist_customers                   |
| olist_geolocation_dataset.csv         | raw.olist_geolocation                 |
| olist_order_items_dataset.csv         | raw.olist_order_items                 |
| olist_order_payments_dataset.csv      | raw.olist_order_payments              |
| olist_order_reviews_dataset.csv       | raw.olist_order_reviews               |
| olist_orders_dataset.csv              | raw.olist_orders                      |
| olist_products_dataset.csv            | raw.olist_products                    |
| olist_sellers_dataset.csv             | raw.olist_sellers                     |
| product_category_name_translation.csv | raw.product_category_name_translation |

## 4 Evidências para os Tipos Físicos

Os arquivos foram perfilados antes da definição dos tipos físicos.

A leitura inicial foi realizada sem inferência automática de tipos, permitindo avaliar os valores como originalmente representados.

Foram examinadas:

- nulidade;
- comprimento máximo;
- domínios categóricos;
- valores mínimos e máximos;
- quantidade de casas decimais;
- formato temporal;
- unicidade;
- integridade referencial;
- características dos identificadores.

| Domínio         | Evidência                  | Decisão                              |
|-----------------|----------------------------|--------------------------------------|
| Identificadores | 32 caracteres hexadecimais | varchar(32); não converter para UUID |
| Prefixo de CEP  | exatamente cinco dígitos   | char(5) com validação                |



| Domínio            | Evidência                                                                        | Decisão                                          |
|--------------------|----------------------------------------------------------------------------------|--------------------------------------------------|
| UF                 | dois caracteres e domínio correspondente às unidades federativas brasileiras     | char(2) com validação de domínio                 |
| Cidades            | maior valor observado com 40 caracteres                                          | varchar(50)                                      |
| Categorias         | maior valor observado com 46 caracteres                                          | varchar(50)                                      |
| Valores monetários | duas casas decimais; maior valor observado de 13.664,08                          | numeric(12,2)                                    |
| Datas e horas      | formato temporal válido sem deslocamento UTC explícito                           | timestamp without time zone                      |
| Coordenadas        | precisão variável e presença de pontos fora dos limites territoriais brasileiros | double precision com limites geográficos globais |

#### 4.1 Decisões específicas de representação

Os identificadores do dataset possuem 32 caracteres hexadecimais, mas não são armazenados como `uuid`. A representação física preserva a característica observada na fonte e evita impor uma semântica técnica que o dataset não declara.

Os prefixos de CEP são atributos textuais com cinco posições. Consequentemente, zeros à esquerda possuem significado estrutural e não devem ser removidos.

Caso alguma etapa intermediária interprete o CEP como valor numérico, sua conversão para o CORE deverá restaurar as cinco posições antes da validação.

Não são impostos valores padrão artificiais para atributos ausentes.

As ausências observadas no dataset permanecem representadas por `NULL` quando semanticamente aceitas pelo modelo.

## 5 Dicionário Físico do CORE

O dicionário a seguir descreve a materialização física das cinquenta colunas do CORE.

As colunas indicadas como obrigatórias utilizam `NOT NULL`.

Nenhum atributo de negócio recebe valor padrão implícito.

Somente `geolocalizacao.id_geolocalizacao`, que não possui equivalente na fonte, é gerado pelo banco como:

```
bigint generated always as identity
```

### 5.1 Tabela cliente

Fonte principal:

`olist_customers_dataset.csv`

Granularidade:

**uma ocorrência de cliente associada ao pedido no dataset.**

| Coluna CORE      | Coluna de origem         | Tipo        | Nulo | Geração / padrão | Integridade e observações                                   |
|------------------|--------------------------|-------------|------|------------------|-------------------------------------------------------------|
| id_cliente       | customer_id              | varchar(32) | Não  | —                | PK; exatamente 32 caracteres hexadecimais                   |
| id_cliente_unico | customer_unique_id       | varchar(32) | Não  | —                | exatamente 32 caracteres hexadecimais                       |
| prefixo_cep      | customer_zip_code_prefix | char(5)     | Não  | —                | FK para <code>prefixo_cep</code> ; exatamente cinco dígitos |
| cidade           | customer_city            | varchar(50) | Não  | —                | texto não vazio                                             |
| estado           | customer_state           | char(2)     | Não  | —                | domínio das unidades federativas brasileiras                |

### 5.2 Tabela pedido

Fonte principal:

`olist_orders_dataset.csv`

Granularidade:

**um pedido.**

| Coluna CORE              | Coluna de origem              | Tipo                        | Nulo | Padrão | Integridade e observações                                        |
|--------------------------|-------------------------------|-----------------------------|------|--------|------------------------------------------------------------------|
| id_pedido                | order_id                      | varchar(32)                 | Não  | —      | PK                                                               |
| id_cliente               | customer_id                   | varchar(32)                 | Não  | —      | FK para cliente; restrição UNIQUE                                |
| status_pedido            | order_status                  | varchar(20)                 | Não  | —      | valor pertencente ao domínio de estados observados no dataset    |
| data_compra              | order_purchase_timestamp      | timestamp without time zone | Não  | —      | instante de criação do pedido                                    |
| data_aprovacao           | order_approved_at             | timestamp without time zone | Sim  | —      | ausência preservada quando não registrada pela fonte             |
| data_envio_transportador | order_delivered_carrier_date  | timestamp without time zone | Sim  | —      | ausência preservada quando não registrada                        |
| data_entrega             | order_delivered_customer_date | timestamp without time zone | Sim  | —      | ausência preservada para pedidos não entregues ou sem informação |
| data_entrega_estimada    | order_estimated_delivery_date | timestamp without time zone | Não  | —      | data estimada registrada no pedido                               |

Não são estabelecidas comparações temporais universais entre todas as datas do ciclo do pedido.

A existência de pedidos cancelados, indisponíveis ou sem determinados eventos registrados torna inadequado impor uma única sequência temporal obrigatória a todos os registros.

### 5.3 Tabela `item_pedido`

Fonte principal:

`olist_order_items_dataset.csv`

Granularidade:

**um item dentro de um pedido.**

| Coluna CORE       | Coluna origem       | Tipo                        | Nulo | Padrão | Integridade                                |
|-------------------|---------------------|-----------------------------|------|--------|--------------------------------------------|
| id_pedido         | order_id            | varchar(32)                 | Não  | —      | parte da PK composta e FK para pedido      |
| id_item           | order_item_id       | smallint                    | Não  | —      | parte da PK composta; valor maior que zero |
| id_produto        | product_id          | varchar(32)                 | Não  | —      | FK para produto                            |
| id_vendedor       | seller_id           | varchar(32)                 | Não  | —      | FK para vendedor                           |
| data_limite_envio | shipping_limit_date | timestamp without time zone | Não  | —      | data limite operacional associada ao item  |
| preco_item        | price               | numeric(12,2)               | Não  | —      | valor maior ou igual a zero                |
| valor_frete       | freight_value       | numeric(12,2)               | Não  | —      | valor maior ou igual a zero                |

### 5.4 Tabela produto

Fonte principal:

`olist_products_dataset.csv`

Granularidade:

**um produto.**



| Coluna CORE           | Coluna origem              | Tipo        | Nulo | Padrão | Integridade e observações          |
|-----------------------|----------------------------|-------------|------|--------|------------------------------------|
| id_produto            | product_id                 | varchar(32) | Não  | —      | PK                                 |
| nome_categoria        | product_category_name      | varchar(50) | Sim  | —      | ausência da categoria é preservada |
| comprimento_nome      | product_name_lenght        | smallint    | Sim  | —      | valor não negativo                 |
| comprimento_descricao | product_description_lenght | smallint    | Sim  | —      | valor não negativo                 |
| quantidade_fotos      | product_photos_qty         | smallint    | Sim  | —      | valor não negativo                 |
| peso_g                | product_weight_g           | integer     | Sim  | —      | valor não negativo                 |
| comprimento_cm        | product_length_cm          | integer     | Sim  | —      | valor não negativo                 |
| altura_cm             | product_height_cm          | integer     | Sim  | —      | valor não negativo                 |
| largura_cm            | product_width_cm           | integer     | Sim  | —      | valor não negativo                 |

O perfilamento identificou ausências simultâneas em atributos descritivos de produtos e ocorrências ausentes em atributos de dimensão física.

Esses valores não são substituídos por zero ou por qualquer outro valor artificial.

A ausência permanece explicitamente representada por `NULL`, pois zero seria semanticamente diferente de informação desconhecida.

### 5.5 Tabela vendedor

Fonte principal:

`olist_sellers_dataset.csv`

Granularidade:

**um vendedor.**

| Coluna CORE | Coluna origem          | Tipo        | Nulo | Padrão | Integridade                                  |
|-------------|------------------------|-------------|------|--------|----------------------------------------------|
| id_vendedor | seller_id              | varchar(32) | Não  | —      | PK                                           |
| prefixo_cep | seller_zip_code_prefix | char(5)     | Não  | —      | FK para prefixo_cep; cinco dígitos           |
| cidade      | seller_city            | varchar(50) | Não  | —      | texto não vazio                              |
| estado      | seller_state           | char(2)     | Não  | —      | domínio das unidades federativas brasileiras |

## 5.6 Tabela pagamento

Fonte principal:

`olist_order_payments_dataset.csv`

Granularidade:

**uma ocorrência de pagamento dentro de um pedido.**

| Coluna CORE          | Coluna origem        | Tipo          | Nulo | Padrão | Integridade                                |
|----------------------|----------------------|---------------|------|--------|--------------------------------------------|
| id_pedido            | order_id             | varchar(32)   | Não  | —      | parte da PK composta e FK para pedido      |
| sequencial_pagamento | payment_sequential   | smallint      | Não  | —      | parte da PK composta; valor maior que zero |
| tipo_pagamento       | payment_type         | varchar(20)   | Não  | —      | domínio de formas de pagamento observadas  |
| numero_parcelas      | payment_installments | smallint      | Não  | —      | valor maior ou igual a zero                |
| valor_pagamento      | payment_value        | numeric(12,2) | Não  | —      | valor maior ou igual a zero                |

O valor zero em `numero_parcelas` não é automaticamente corrigido ou descartado.

A fonte contém essa ocorrência e ela pode representar uma condição de pagamento sem parcelamento segundo a codificação do dataset.

Qualquer reinterpretação desse valor deverá ocorrer como regra de transformação explicitamente documentada, e não como correção silenciosa do dado físico.

### 5.7 Tabela avaliacao

Fonte principal:

`olist_order_reviews_dataset.csv`

Granularidade:

**uma avaliação associada a um pedido.**

| Coluna CORE    | Coluna origem           | Tipo                        | Nulo | Padrão | Integridade                           |
|----------------|-------------------------|-----------------------------|------|--------|---------------------------------------|
| id_avaliacao   | review_id               | varchar(32)                 | Não  | —      | parte da PK composta                  |
| id_pedido      | order_id                | varchar(32)                 | Não  | —      | parte da PK composta e FK para pedido |
| nota_avaliacao | review_score            | smallint                    | Não  | —      | valor entre 1 e 5                     |
| data_criacao   | review_creation_date    | timestamp without time zone | Não  | —      | data de criação da avaliação          |
| data_resposta  | review_answer_timestamp | timestamp without time zone | Não  | —      | instante registrado para resposta     |

Os campos textuais de título e comentário existentes no arquivo de origem não são materializados no CORE porque não pertencem ao modelo lógico aprovado.

Eles permanecem disponíveis na RAW, preservando a possibilidade de incorporação futura caso o escopo analítico passe a exigir análise textual.

## 5.8 Tabela prefixo\_cep

A tabela consolida os prefixos de CEP utilizados para relacionar clientes, vendedores e geolocalizações.

| Coluna      | Tipo    | Nulo | Integridade                  |
|-------------|---------|------|------------------------------|
| prefixo_cep | char(5) | Não  | PK; exatamente cinco dígitos |

## 5.9 Tabela geolocalizacao

Fonte principal:

`olist_geolocation_dataset.csv`

Granularidade:

**uma ocorrência geográfica observada no dataset.**

O arquivo pode apresentar mais de uma coordenada associada ao mesmo prefixo de CEP. Por esse motivo, o prefixo não identifica individualmente uma linha de geolocalização.

Foi criada uma chave substituta estável para a tabela.



| Coluna CORE       | Coluna origem               | Tipo             | Nulo | Geração                      | Integridade                                  |
|-------------------|-----------------------------|------------------|------|------------------------------|----------------------------------------------|
| id_geolocalizacao | —                           | bigint           | Não  | generated always as identity | PK substituta                                |
| prefixo_cep       | geolocation_zip_code_prefix | char(5)          | Não  | —                            | FK para prefixo_cep                          |
| latitude          | geolocation_lat             | double precision | Não  | —                            | entre -90 e 90                               |
| longitude         | geolocation_lng             | double precision | Não  | —                            | entre -180 e 180                             |
| cidade            | geolocation_city            | varchar(50)      | Não  | —                            | texto não vazio                              |
| estado            | geolocation_state           | char(2)          | Não  | —                            | domínio das unidades federativas brasileiras |

Os limites de latitude e longitude são globais e não restritos ao território brasileiro.

Essa decisão preserva ocorrências existentes na própria fonte que podem se localizar fora dos limites territoriais esperados sem tornar a carga fisicamente impossível.

## 6 Política de Integridade Referencial

As relações implementadas no CORE são:

- `pedido.id_cliente → cliente.id_cliente;`
- `item_pedido.id_pedido → pedido.id_pedido;`
- `item_pedido.id_produto → produto.id_produto;`
- `item_pedido.id_vendedor → vendedor.id_vendedor;`
- `pagamento.id_pedido → pedido.id_pedido;`
- `avaliacao.id_pedido → pedido.id_pedido;`
- `cliente.prefixo_cep → prefixo_cep.prefixo_cep;`
- `vendedor.prefixo_cep → prefixo_cep.prefixo_cep;`
- `geolocalizacao.prefixo_cep → prefixo_cep.prefixo_cep.`

Todas as exclusões referenciadas utilizam:

ON DELETE RESTRICT

As atualizações dos identificadores referenciados utilizam:

ON UPDATE NO ACTION

Não são utilizadas exclusões em cascata.

A remoção automática de registros relacionados poderia eliminar informações históricas de outras entidades sem uma decisão explícita do processo de dados.

## 7 Estratégia Inicial de Índices

A estratégia de indexação foi definida de forma conservadora.

Índices somente são adicionados quando derivados diretamente de restrições estruturais ou de relacionamentos cuja navegação possui utilidade clara.

O PostgreSQL cria automaticamente índices B-tree para:

- chaves primárias;
- restrições UNIQUE.

Entretanto, o PostgreSQL não cria automaticamente índices para as colunas que apenas participam como lado referenciante de uma chave estrangeira.

Uma chave estrangeira é considerada coberta quando suas colunas ocupam o prefixo à esquerda de algum índice existente.

### 7.1 Cobertura gerada pelas restrições

| Tabela         | Índice implícito                     | Cobertura relevante                    |
|----------------|--------------------------------------|----------------------------------------|
| cliente        | PK (id_cliente)                      | identificação de cliente               |
| pedido         | PK (id_pedido)                       | identificação de pedido                |
| pedido         | UNIQUE (id_cliente)                  | FK pedido → cliente                    |
| item_pedido    | PK (id_pedido, id_item)              | FK item → pedido                       |
| produto        | PK (id_produto)                      | identificação de produto               |
| vendedor       | PK (id_vendedor)                     | identificação de vendedor              |
| pagamento      | PK (id_pedido, sequencial_pagamento) | FK pagamento → pedido                  |
| avaliacao      | PK (id_avaliacao, id_pedido)         | não cobre busca iniciada por id_pedido |
| prefixo_cep    | PK (prefixo_cep)                     | identificação do prefixo               |
| geolocalizacao | PK (id_geolocalizacao)               | identificação da ocorrência geográfica |

### 7.2 Índices adicionais do CORE

Foram definidos seis índices B-tree adicionais:

- `idx_cliente_prefixo_cep` sobre `cliente(prefixo_cep)`;
- `idx_vendedor_prefixo_cep` sobre `vendedor(prefixo_cep)`;
- `idx_item_pedido_id_produto` sobre `item_pedido(id_produto)`;
- `idx_item_pedido_id_vendedor` sobre `item_pedido(id_vendedor)`;

- `idx_avaliacao_id_pedido` sobre `avaliacao(id_pedido)`;
- `idx_geolocalizacao_prefixo_cep` sobre `geolocalizacao(prefixo_cep)`.

Esses índices correspondem às chaves estrangeiras que não estão adequadamente cobertas pelo prefixo de uma chave primária ou restrição UNIQUE existente.

### 7.3 RAW

Nenhum índice secundário adicional é inicialmente criado na camada RAW.

O objetivo dessa camada é priorizar simplicidade e desempenho de ingestão.

Adicionar índices antes da existência de padrões reais de consulta aumentaria o custo de escrita sem benefício comprovado.

Cada tabela possui apenas o índice decorrente de sua chave primária técnica.

### 7.4 ANALYTICS

Nenhum índice é criado inicialmente no schema ANALYTICS porque ele ainda não contém objetos analíticos.

A estratégia deverá ser definida individualmente para cada estrutura que venha a ser criada.

### 7.5 Revisão posterior

Os índices iniciais não são considerados definitivos.

Após a carga e o início das consultas deverão ser avaliados:

- estatísticas do PostgreSQL;
- consultas recorrentes;
- seletividade;
- custo de manutenção;
- planos obtidos com EXPLAIN;
- planos obtidos com EXPLAIN ANALYZE.

Índices adicionais somente deverão ser criados com evidência de uso.

## 8 Scripts da Arquitetura Física

A implementação é representada por scripts SQL versionados.

Os principais artefatos são:

- `create_schema.sql`  
Cria os schemas RAW, CORE e ANALYTICS, bem como suas respectivas tabelas e restrições;
- `create_indexes.sql`  
Cria os seis índices secundários adicionais definidos para o CORE;
- `validate_schema.sql`  
Valida propriedades estruturais do catálogo e o comportamento das principais restrições;
- `drop_indexes.sql`  
Remove explicitamente os índices secundários adicionais;
- `drop_schema.sql`  
Remove as estruturas físicas em ordem segura.

### 8.1 Execução pelo DBeaver

Para criar e validar a arquitetura em uma instância limpa:

1. conectar-se ao servidor PostgreSQL;
2. abrir um editor SQL associado ao banco desejado;
3. executar `create_schema.sql`;
4. confirmar a criação dos schemas e tabelas;
5. executar `create_indexes.sql`;
6. executar `validate_schema.sql`;
7. verificar se a validação foi concluída sem falhas.

A criação da estrutura não utiliza redefinição silenciosa de objetos.

Se os schemas já existirem, uma nova execução do script de criação deverá falhar em vez de substituir estruturas existentes.

Essa decisão reduz o risco de alterações involuntárias em um banco já criado.

### 8.2 Remoção

Os scripts de remoção devem ser utilizados somente:

- em ambientes descartáveis;
- em testes automatizados;
- quando houver decisão explícita de reconstruir o banco.

Uma sequência possível é:

1. executar `drop_indexes.sql`;
2. executar `drop_schema.sql`.

A operação de remoção foi projetada para poder ser repetida sem deixar o ambiente em estado intermediário inconsistente.

## 9 Validação da Arquitetura

A arquitetura física foi testada em uma instância limpa e descartável do **PostgreSQL 18.3**.

O ambiente de teste não dependeu de configurações particulares do DBeaver, garantindo que a validação estivesse associada aos scripts SQL e ao PostgreSQL.

### 9.1 Procedimento de validação

A validação contemplou:

1. identificação da versão do PostgreSQL utilizada;
2. execução do script de criação da arquitetura;
3. inspeção dos schemas criados;
4. validação da quantidade de tabelas e colunas;
5. validação de chaves primárias, estrangeiras e restrições UNIQUE;
6. criação e contagem dos índices;
7. teste da permissividade intencional da RAW;

8. teste das restrições de integridade do CORE;
9. verificação de rollback dos registros utilizados nos testes;
10. teste de criação repetida e remoção repetida da arquitetura.

## 9.2 Validação estrutural

| Verificação                    | Esperado | Obtido |
|--------------------------------|----------|--------|
| Versão principal do PostgreSQL | 18       | 18     |
| Schemas físicos                | 3        | 3      |
| Tabelas RAW                    | 9        | 9      |
| Tabelas CORE                   | 9        | 9      |
| Objetos ANALYTICS              | 0        | 0      |
| Colunas RAW                    | 79       | 79     |
| Colunas CORE                   | 50       | 50     |
| Colunas identity               | 10       | 10     |
| Chaves primárias               | 18       | 18     |
| Chaves estrangeiras CORE       | 9        | 9      |
| Restrições UNIQUE CORE         | 1        | 1      |
| Índices RAW                    | 9        | 9      |
| Índices CORE totais            | 16       | 16     |
| Índices adicionais CORE        | 6        | 6      |

## 9.3 Validação da integridade referencial

As nove chaves estrangeiras do CORE foram verificadas.

Todas apresentaram:

ON DELETE RESTRICT

e:

ON UPDATE NO ACTION

confirmando a política de preservação dos registros relacionados.

### 9.4 Teste da camada RAW

A RAW foi testada com valores que seriam inválidos segundo as regras do CORE.

A inserção foi aceita conforme esperado.

Isso confirma que a camada não está aplicando antecipadamente regras de negócio sobre os valores recebidos.

Sua função permanece sendo a preservação da evidência original da fonte.

### 9.5 Teste da camada CORE

O CORE foi submetido a inserções deliberadamente inválidas.

Foram testados, entre outros comportamentos:

- prefixo de CEP incompatível com o domínio;
- tentativa de duplicação de chave primária;
- tentativa de criação de referência órfã.

As operações foram rejeitadas conforme esperado.

Isso confirma que as regras de integridade estão efetivamente sendo aplicadas pelo PostgreSQL e não apenas documentadas.

### 9.6 Isolamento dos testes

Os registros utilizados para testar as restrições foram executados dentro de transações e posteriormente revertidos.

Dessa forma, a execução da validação não deixa registros fictícios incorporados ao banco.

### 9.7 Repetibilidade da criação

Após uma primeira criação bem-sucedida, o script de criação foi executado novamente.

A segunda tentativa foi rejeitada intencionalmente porque os schemas já existiam.

As dezoito tabelas previamente criadas permaneceram intactas.

Esse comportamento protege o ambiente contra redefinições silenciosas.



### 9.8 Repetibilidade da remoção

A rotina de remoção foi executada mais de uma vez.

Após a primeira remoção, uma nova execução não produziu falha decorrente da inexistência dos objetos.

Esse comportamento permite reconstruir ambientes descartáveis de forma previsível.

### 9.9 Resultado

Os testes confirmaram:

- criação correta dos três schemas;
- criação das nove tabelas RAW;
- criação das nove tabelas CORE;
- ausência intencional de objetos ANALYTICS;
- correspondência das colunas esperadas;
- funcionamento das chaves e restrições;
- criação dos índices previstos;
- permissividade controlada da RAW;
- integridade efetiva do CORE;
- segurança transacional dos testes;
- comportamento previsível de criação e remoção.

A arquitetura física é, portanto, considerada **aprovada para receber o processo de carga e transformação dos dados**.

## 10 Responsabilidades da Carga e Transformação

A modelagem física define as estruturas que deverão receber os dados, mas não antecipa todas as regras operacionais do processo de carga.

O processo subsequente deverá:

- extrair os nove arquivos CSV;
- carregar os registros na RAW com rastreabilidade;
- converter tipos físicos;
- normalizar prefixos de CEP;
- validar domínios;
- tratar duplicidades quando necessário;
- construir as entidades do CORE;
- respeitar a ordem necessária para as chaves estrangeiras;
- contabilizar registros recebidos, carregados, transformados e rejeitados;
- reconciliar os volumes entre fonte, RAW e CORE.

Regras específicas de transformação deverão permanecer separadas das restrições puramente estruturais.

Isso evita introduzir no modelo físico interpretações analíticas ou correções que ainda não tenham sido justificadas.

## 11 Preparação para Consumo Analítico

Depois da carga validada do CORE, consultas analíticas poderão ser utilizadas para verificar se o modelo atende às necessidades funcionais previstas.

Quando uma transformação ou agregação passar a ser reutilizada por diversas análises, poderá ser materializada no schema ANALYTICS.

Entre as possíveis estruturas futuras estão:

- visão analítica de pedidos;
- métricas de vendas;
- métricas de pagamentos;
- indicadores de entrega;
- análises por produto e categoria;
- análises por vendedor;

- análises geográficas;
- métricas de avaliações;
- estruturas específicas para Power BI.

A criação dessas estruturas não altera o papel do CORE como camada relacional íntegra.

## 12 Conclusão

A modelagem física transforma o modelo lógico em uma implementação relacional reproduzível no PostgreSQL.

A arquitetura separa claramente três responsabilidades:

- **RAW:** preservação rastreável dos dados recebidos;
- **CORE:** representação relacional tipada, validada e íntegra;
- **ANALYTICS:** estruturas derivadas destinadas ao consumo analítico.

O PostgreSQL 18 foi adotado como SGBD, enquanto o DBeaver atua como cliente gráfico de apoio à administração e execução dos scripts.

Os tipos físicos foram definidos a partir do perfil observado no dataset, preservando identificadores, valores ausentes, prefixos de CEP, coordenadas e características temporais da fonte.

A estratégia inicial de índices utiliza os índices implícitos das restrições do PostgreSQL e adiciona somente os seis índices necessários para complementar a cobertura das chaves estrangeiras.

Os scripts de criação, indexação, validação e remoção permitem reproduzir e testar a arquitetura sem depender de configuração manual do cliente gráfico.

Os testes realizados confirmaram a estrutura esperada do catálogo, a permissividade intencional da RAW, a aplicação efetiva das regras de integridade no CORE e a segurança das rotinas de criação e remoção.

Com a arquitetura física validada, o banco encontra-se preparado para receber os arquivos do dataset, registrar sua origem na RAW, transformar os dados para o CORE e posteriormente disponibilizar estruturas analíticas para consultas SQL e ferramentas de Business Intelligence.

## 13 Referências

- PostgreSQL. *Versioning Policy*. Disponível em: <https://www.postgresql.org/support/versioning/>.
- PostgreSQL. *PostgreSQL 18 Documentation*. Disponível em: <https://www.postgresql.org/docs/18/>.
- DBeaver. *PostgreSQL driver documentation*. Disponível em: <https://dbeaver.com/docs/dbeaver/Database-driver-PostgreSQL/>.